

A.D. 1308
unipg
UNIVERSITÀ DEGLI STUDI
DI PERUGIA

Tesina Di Algoritmi e Strutture Dati
Corso di Laurea in Ingegneria Informatica ed Elettronica
A.A 2025-2026
Prof. E. Di Giacomo

Implementazione di Algoritmi per il problema del Pattern Matching

A cura di:
Francesca Moretti | matricola: 369131
francesca.moretti2@studenti.unipg.it

Sommario

Sommario.....2
Introduzione e definizione del problema.....3
1.1 Il problema del Pattern Matching.....3
1.2 Una prima soluzione: l'approccio Naive.....3
Implementazione dell'algoritmo di Pattern Matching.....4
2.1 Teoria e Logica del Modello Computazionale.....4
2.2 Funzione di transizione e funzione suffisso.....6
2.3 Pseudocodice e scelte implementative.....6
Analisi della complessità.....8
3.1 Complessità temporale e spaziale - Brute Force.....8
Complessità temporale.....8
Complessità spaziale.....8
3.2 Complessità temporale e spaziale - Automa a Stati Finiti.....8
Complessità temporale.....9
Complessità spaziale.....9
3.3 Confronto tra gli approcci.....9
Dati sperimentali e analisi dei risultati.....9
4.1 Metodologia dei test e dati ottenuti.....9
4.1 Discussione dei dati.....10
Conclusioni.....11

Introduzione e definizione del problema

1.1 Il problema del Pattern Matching

Il problema del Pattern Matching consiste nell'individuare la prima occorrenza di una stringa all'interno di un testo.
L'algoritmo del Pattern Matching è cruciale, basti pensare ai suoi ambiti di applicazione: viene principalmente utilizzato da editor di testo per la ricerca di occorrenze di una parola specifica fornita dall'utente. L'algoritmo ha anche altre fondamentali applicazioni: la ricerca di una particolare sequenza di basi azotate nel DNA, oltre ad essere alla base delle ricerche sul web.

L'obiettivo di questo progetto è sviluppare un algoritmo ottimale di Pattern Matching, seguendo la Traccia 2 del corso Algoritmi e Strutture Dati.

Diamo una definizione formale al problema.
Assumiamo che il testo T sia un array di caratteri di lunghezza n, T[1..n] e la stringa da cercare nel testo sia a sua volta un array di caratteri, P[1..m] di lunghezza m, con m≤n.
Diciamo che la stringa P appare nel testo T con "shift" s (quindi nella posizione s+1 del testo) se T[s+1..s+m] = P[1..m] con 0≤s≤n-m.

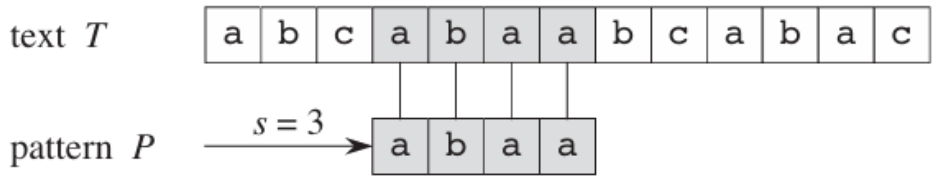


figura 1.1

Nell'esempio la stringa "abaa" appare nel testo alla posizione s+1, cioè in posizione 4.

1.2 Una prima soluzione: l'approccio *Naive*

La prima soluzione che potrebbe venire in mente è quella di considerare ogni "shift" possibile e, per ognuno di questi, confrontare ogni carattere nel testo con ogni carattere della stringa cercata. Appena troviamo un carattere che non coincide il ciclo termina e si passa allo "shift" successivo.

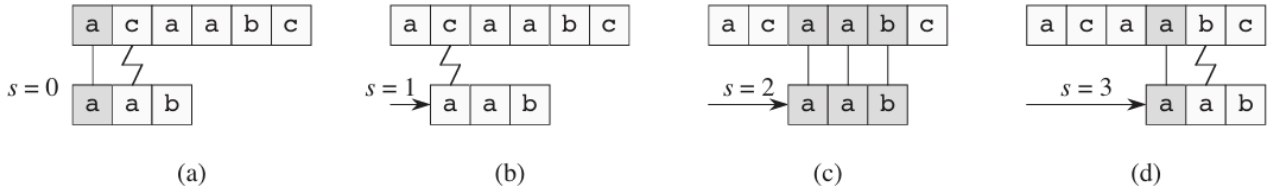


figura 1.2

Nel caso peggiore, il numero di confronti da fare è dato da (n-m+1)*m, dove (n-m+1) è il massimo numero di shift possibili. Ciò significa che, per ogni shift possibile, devo confrontare m caratteri per validare lo shift.
Questo approccio, detto *Naive* (ingenuo) o *Brute Force* (forza bruta) presenta un limite importante: ogni volta che c'è un *mismatch* (i due caratteri non corrispondono), il controllo ricomincia da capo, perdendo tutte le informazioni acquisite precedentemente.

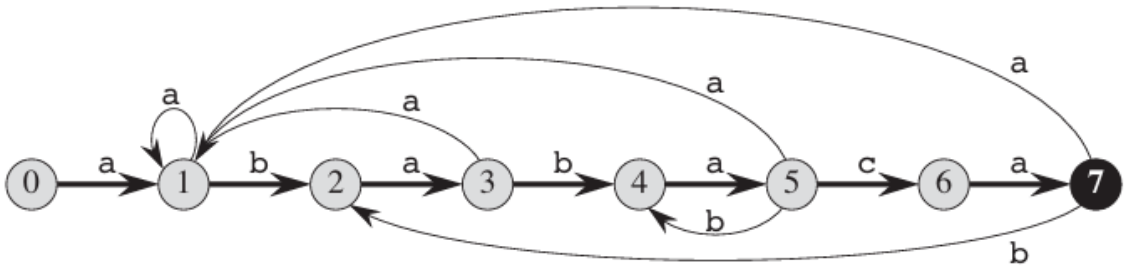
Implementazione dell'algoritmo di Pattern Matching

2.1 Teoria e Logica del Modello Computazionale

La soluzione ottimale per il problema di Pattern Matching si ottiene grazie alla costruzione di automi a stati finiti, questi operano in modo molto efficiente, esaminando ogni carattere solamente una volta. La complessità dell'algoritmo quindi dipende solo dalla lunghezza del testo da esaminare.
L'algoritmo, tuttavia, richiede un certo tempo di preprocessing per la costruzione di una funzione suffisso, il cui compito verrà spiegato nel prossimo paragrafo, che nel caso del Brute Force non era previsto.

Un automa a stati finiti è definito come la quintupla $(Q, q_0, A, \Sigma, \delta)$ in cui:

- Q è l'insieme degli stati
- $q_0 \in Q$ è lo stato iniziale
- $A \subseteq Q$ è l'insieme degli stati di accettazione
- Σ è l'alfabeto da considerare
- δ è la funzione di transizione dell'automa



(a)

figura 2.1

Nell'esempio è stato implementato un automa a stati finiti per riconoscere la sequenza P="`ababaca`". Lo stato iniziale è q_0=0, l'unico stato di accettazione è A=7 poiché per riconoscere la sequenza devo arrivare allo stato 7.
Per rendere più chiaro come funziona l'automa a stati, pensiamo ad ogni stato in questo modo:

- stato 0: non ho ricevuto niente
- stato 1: ho ricevuto "`a`", se dallo stato corrente ricevo "`a`", rimango nello stato di ricezione di una sola "`a`". Se arriva "`b`" proseguo allo stato successivo
- stato 2: ho ricevuto "`ab`", procedo allo stato successivo solo se ricevo "`a`", in entrambi i casi di ricezione di "`b`" o "`c`" si torna allo stato iniziale 0, non ho ricevuto nulla

E così via.

Dal diagramma a stati sopra riportato si costruisce una funzione delle transizioni dell'automa.

	input			
state	a	b	c	*P*
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

(b)

figura 2.2

2.2 Funzione di transizione e funzione suffisso

Per definire la funzione di transizione ci basiamo su una funzione ausiliaria chiamata funzione suffisso σ(x) definita come segue:

$$\sigma(x)=\max\{k: P_k \text{ suffisso di } x\}$$

cioè la lunghezza massima di P che compare esattamente alla fine di x.
Facciamo un esempio:
P=″abababaca″
x=″abab″
i possibili prefissi di P sono:

- P_0=stringa vuota
- P_1=″a″
- P_2=″ab″
- P_3=″aba″
- P_4=″abab″

e così via.

Quindi σ(“abab″)=4.

Definiamo a questo punto la funzione di transizione come segue:

$$\delta(q, a) = \sigma(P_q a)$$

dove q è lo stato e a è il carattere in input (vedi *figura 2.2*).
Capire il significato di $\sigma(P_q a)$ è il punto cruciale dell’algoritmo, si tratta della lunghezza del prefisso più lungo di P che sia anche suffisso della stringa $P_q a$.
P_q è la stringa formata dai primi q caratteri indovinati fino a quel momento, come dicevamo prima, se siamo nello stato 2 significa che l’automa ha già indovinato con successo 2 caratteri di P.
a è il nuovo carattere appena letto dal testo.

2.3 Pseudocodice e scelte implementative

Vediamo prima lo pseudocodice per la costruzione della funzione di transizione.

```
COMPUTE-TRANSITION-FUNCTION(P, Σ)
1  m = P.length
2  for q = 0 to m
3      for each character a ∈ Σ
4          k = min(m + 1, q + 2)
5          repeat
6              k = k − 1
7          until P_k ⊐ P_q a
8          δ(q, a) = k
9  return δ
```

figura 2.3

La scelta implementativa più rilevante è la rappresentazione della funzione di transizione tramite una matrice bidimensionale di interi di dimensione m+1 (che indica il numero degli stati) * 256.
Lo pseudocodice prevede un ciclo che itera su ogni carattere dell'alfabeto Σ del pattern, tuttavia si è optato per l'utilizzo dell'intero set di caratteri ASCII per due motivi

- evitare errori nel caso in cui l'algoritmo trovasse un carattere non compreso nell'alfabeto Σ
- utilizzare il carattere letto dal testo direttamente come indice delle colonne di δ[q][c]

Per il calcolo di $\sigma(P_q a)$ è stato utilizzato il metodo .endsWith() di Java che verifica se una stringa termina con i caratteri specificati.
In particolare, per il calcolo delle nuove stringhe si utilizza un particolare costruttore della classe String di Java:
`String Pqa = new String(P, 0, q) + c` - crea una nuova stringa $P_q a$ che copia i caratteri di P da 0 a q, cioè tutti i caratteri indovinati, e aggiunge il nuovo carattere appena letto dal testo.
`String pk = new String(P, 0, k)` - crea una nuova stringa prefisso candidata ad essere suffisso di $P_q a$.

Una volta costruita la funzione di transizione, l'algoritmo è praticamente completato, in quanto basta scorrere tutti i caratteri presenti nel testo e modificare lo stato in base a quello che leggiamo nella funzione di transizione q=δ[q][T[i]], che significa "se mi trovo allo stato q e mi arriva il carattere T[i], aggiorna lo stato q con il valore della matrice in quel punto".

```
FINITE-AUTOMATON-MATCHER(T, δ, m)
1  n = T.length
2  q = 0
3  for i = 1 to n
4       q = δ(q, T[i])
5       if q == m
6            print "Pattern occurs with shift" i − m
```

figura 2.4

Nell'implementazione del codice si fa un ulteriore controllo sui caratteri che verifica che rientrino nei 256 della set ASCII, in caso contrario q viene azzerato (ricordiamo che q indica il numero di caratteri indovinati, cioè lo stato in cui ci troviamo in quel momento).

Analisi della complessità

L'obiettivo di questo capitolo è analizzare la complessità computazionale, in termini di spazio e memoria, dei due algoritmi implementati.

3.1 Complessità temporale e spaziale - Brute Force

L'algoritmo, come già detto, non richiede nessuna fase di preprocessing.

Complessità temporale

I confronti nel caso peggiore sono il numero di shift possibili per la lunghezza del pattern quindi O((n-m+1)m). Realisticamente si assume che $m \ll n$ quindi la complessità diventa $\Theta(n*m)$.
Nel caso migliore l'algoritmo incontra un mismatch al primo carattere della stringa, dopo un solo confronto il ciclo si interrompe, la complessità si riconduce ad un andamento lineare $\Theta(n)$.

Complessità spaziale

L'algoritmo non richiede strutture dati aggiuntive quindi la complessità spaziale è costante $\Theta(1)$.

3.2 Complessità temporale e spaziale - Automa a Stati Finiti

In questo caso è richiesta una fase di preprocessing che coincide con la costruzione della funzione di transizione.

Complessità temporale

1. Calcolo della funzione di transizione $\Theta(m^3|\Sigma|)$: il ciclo esterno scorre tutti gli stati da 0 a m, il ciclo interno scorre tutti i caratteri dell'alfabeto $|\Sigma|$, il while scorre a ritroso per verificare se il prefisso candidato è suffisso, il confronto tra le due stringhe viene effettuato al più m volte.
2. Fase di matching $\Theta(n)$: il ciclo for scorre tutti i caratteri del testo una sola volta.

Complessità spaziale

L'automa memorizza la funzione di transizione in una matrice di dimensione $(m+1)*|\Sigma|$, nel nostro caso la dimensione dell'alfabeto è 256, quindi lo spazio occupato in memoria è $\Theta(m*|\Sigma|)$.

3.3 Confronto tra gli approcci

Algoritmo	**Tempo processing**	**Complessità temporale**	**Complessità spaziale**
Brute Force	nullo	$\Theta(n*m)$	$\Theta(1)$
Automa a stati finiti	$\Theta(m^3\|\Sigma\|)$	$\Theta(n)$	$\Theta(m*\|\Sigma\|)$

Tabella 3.1

Dati sperimentali e analisi dei risultati

Sono stati effettuati dei test per analizzare la differenza di prestazioni dei due algoritmi.

4.1 Metodologia dei test e dati ottenuti

Il primo test è stato effettuato sul testo *I Promessi Sposi*, prima considerando 300.000 caratteri, poi 600.000 e infine il testo completo, si richiede di trovare una parola che non appare nel testo, cioè "smartphone".
Analogamente un secondo test è stato fatto su un testo in grado di mettere alla prova gli algoritmi, un file di sole "a" in cui si richiede di trovare la sequenza "aaab", la "b" è esattamente l'ultimo carattere del testo.
Anche in questo caso prima consideriamo 300.000 caratteri, poi 600.000 e infine tutto il testo (circa 1 milione di caratteri).

I risultati dei test sono mostrati sul terminale e sotto riportati:

```
================================================================================
                              RISULTATI TEST
================================================================================
Test                    Tempo BF        Tempo FA        Confronti BF    Confronti FA
--------------------------------------------------------------------------------
Promessi Sposi 300.000  9,1302          5,7357          382550          366906
Promessi Sposi 600.000  5,9779          4,2689          681750          654287
Promessi Sposi Completo 3,3757          4,4199          1365555         1310306
Caso Peggiore 300.000   2,6134          1,6763          1882538         324358
Caso Peggiore 600.000   6,9293          2,7306          3515866         605776
Caso Peggiore completo  14,1467         4,1291          6393578         1101598
================================================================================
```

figura 4.1

4.1 Discussione dei dati

I risultati dei test sono stati riportati in due grafici, che andiamo ad analizzare.

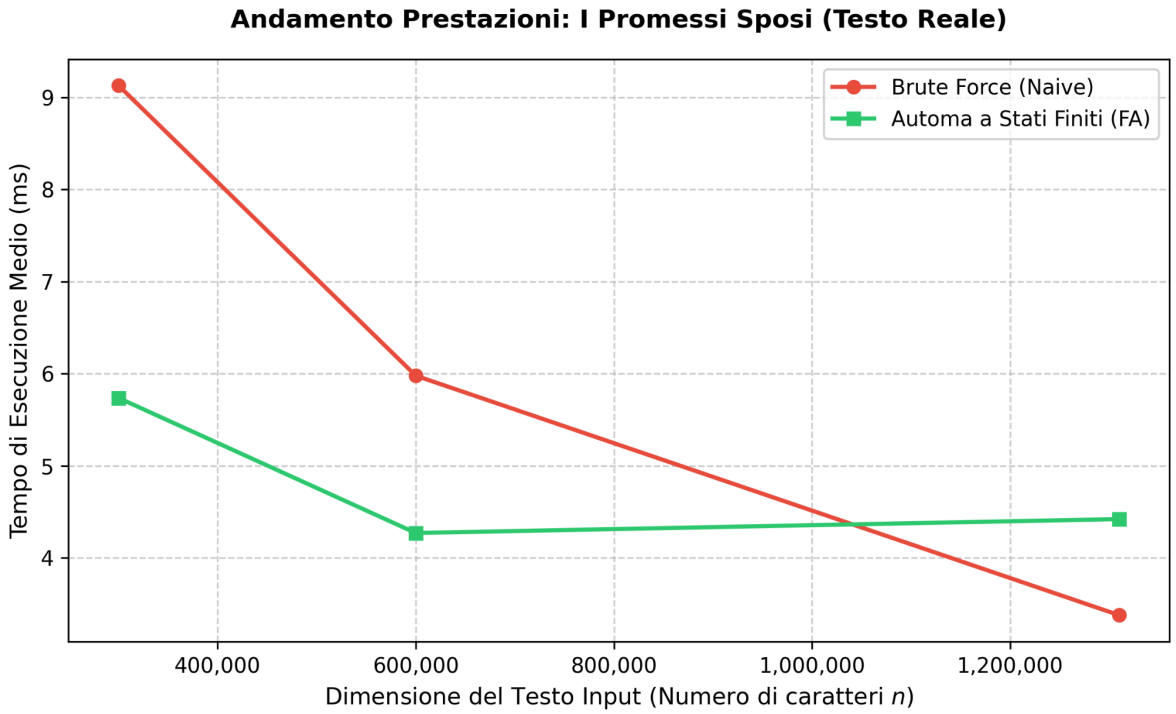


figura 4.2

Dal grafico emergono alcune considerazioni importanti:

- i tempi diminuiscono passando da 300.000 caratteri a 600.000, questo potrebbe essere dovuto dal compiler di Java che richiede un certo tempo di warm-up.
- i tempi dei due algoritmi si differenziano di poco o, in alcuni casi, il Brute Force impiega meno tempo, anche con input grandi. Questo è dovuto dal fatto che nei testi scritti in linguaggio naturale (come quello

analizzato), la probabilità che le prime lettere del pattern coincidano frequentemente con stringhe casuali del testo è estremamente bassa. Di conseguenza, ci troviamo spesso di fronte ad un'analisi di complessità di caso medio/migliore O(n).

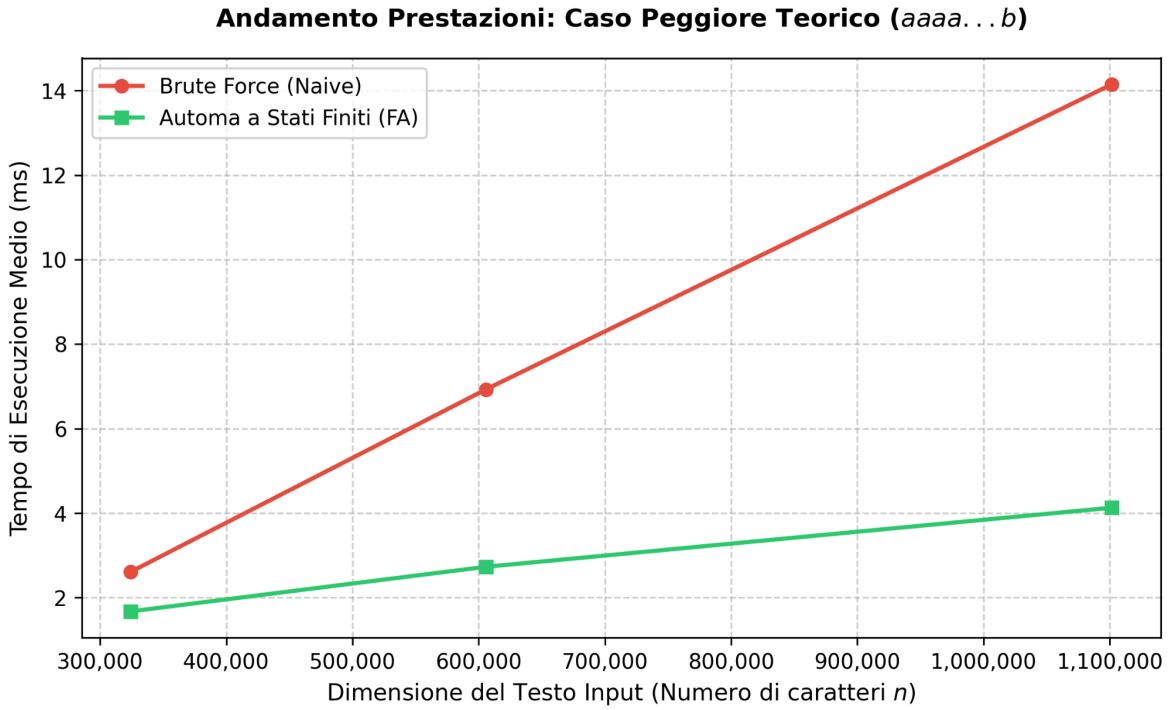


figura 4.3

Nel secondo grafico notiamo uno stacco maggiore tra i due algoritmi, che evidenzia la differenza di complessità molto meglio rispetto al caso precedente. All'aumentare dell'input il tempo del Brute Force aumenta vertiginosamente, mentre quello dell'Automa rimane stabile.

In entrambi i casi, ovviamente, il numero di confronti dell'Automa è nettamente inferiore rispetto al Brute Force.

Conclusioni

L'obiettivo di questo elaborato è stato analizzare, implementare e mettere a confronto due approcci computazionali radicalmente diversi per la risoluzione del problema del Pattern Matching.

I risultati sperimentali hanno confermato i modelli teorici di complessità, offrendo interessanti spunti di riflessione: in contesti reali, i due approcci sono simili in quanto a prestazioni, in contesti critici, lo stress test ha esposto la debolezza del Brute Force rispetto all'Automa.

In conclusione, l'analisi evidenzia il classico compromesso ingegneristico (*trade-off*) tra spazio e tempo. L'Automa a Stati Finiti richiede un investimento iniziale in termini di memoria e tempo di calcolo per la costruzione della matrice di transizione. Tuttavia, tale costo di preprocessing viene ampiamente ammortizzato e si rivela una scelta architetturale vincente quando l'algoritmo opera su flussi di dati massivi o in sistemi di ricerca ripetuta in tempo reale, configurandosi come l'unica soluzione in grado di garantire prestazioni ottimali, costanti e prevedibili in qualunque scenario operativo.